

King Saud University
College of Computer and Information Sciences

Department of Computer Science

CSC 212 Data Structures Project Report – 2nd Semester 2025-2026

Title

Authors

Name	ID	List of all methods implemented by each student
Faris Faleh Saleh Alharbi	445102358	Implemented Classes & Interfaces: • Node.java • LinkedList.java • IStudentList.java / StudentList.java • Main.java • AdvisingSystem.java (<i>File I/O & Helper Methods</i>) List of All Implemented Methods: • main(String[] args) • loadStudentsFromCSV(String path) • loadEventsFromCSV(String path) • getStudentIdsByNames(String namesStr) • insert(T data) • remove() • retrieve() • update(T data) • findFirst() • findNext() • empty() • last() • Node Constructor & Public Attributes (data, next).
Faisal Saud Fahad Alsahli	445100823	Implemented Classes & Interfaces: • Student.java / IStudent.java • Event.java / IEvent.java (<i>Abstract Class & Interface</i>) • Meeting.java / IMeeting.java • Workshop.java / IWorkshop.java • DateTime.java / IDateTime.java • IEventList.java / EventList.java • AdvisingSystem.java (<i>Core System Operations</i>)

		List of All Implemented Methods: • <code>addStudent(Student s)</code> • <code>deleteStudent(int id)</code> • <code>searchStudentById(int id)</code> • <code>scheduleMeeting(String title, DateTime start, DateTime end, String location, int studentId)</code> • <code>scheduleWorkshop(String title, DateTime start, DateTime end, String location, int[] studentIds)</code> • <code>printStudentsByName(String name)</code> • <code>printEventDetailsByTitle(String title)</code> • <code>printEventDetailsByStudentName(String name).</code> • <code>printAllEventsAlphabetically()</code> • <code>addParticipant(Student s)</code> • <code>compareTo(DateTime other)</code> • <code>format()</code> • <code>toString()</code> (Student, Meeting, Workshop).

1. Introduction

[The Student Advising System is an academic platform designed to manage student profiles, schedule individual meetings, and coordinate group workshops efficiently. Built entirely without the standard Java Collections Framework, the system utilizes a custom cursor-based singly linked list to perform stable data operations and avoid schedule conflicts.]

This report covers the overall design architecture of the system, provides a detailed time complexity (Big-O) analysis for its core operations, and outlines the specific implementation methods and workload distribution completed by each team member.]

2. Specification

[

ADT LinkedList

- **Elements:** Generic type T.
- **Structure:** A linear sequence of elements $e_1, e_2, \dots, e_n$ where $n \geq 0$. A current element pointer (cursor) is maintained to point to an active element in the list, or remains undefined.
- **Domain:** Bounded by the available heap memory.
- **Operations:**

findFirst()

- Input: None.
- Process: Resets the current pointer (cursor) to the first element of the list.
- Post-condition: Cursor points to the first element (if the list is not empty).

findNext()

- Input: None.
- Requires: List is not empty, and the cursor is defined and not at the last element.
- Process: Advances the cursor to the next element in the sequence.
- Post-condition: Cursor points to the subsequent node.

retrieve(): T

- Input: None.

- Requires: The cursor is defined.
- Process: Returns the element data currently referenced by the cursor.
- Output: Returns data of type T.

update(T data)

- Input: Element data of type T.
- Requires: The cursor is defined.
- Process: Replaces the data of the current element with the new input data.
- Post-condition: Current element value is modified.

insert(T data)

- Input: Element data of type T.
- Process: Inserts a new element immediately following the current cursor position. If the list is empty, it becomes the head. The cursor then moves to the newly inserted node.
- Post-condition: List size increases by 1; cursor updates to the new element.

remove()

- Input: None.
- Requires: The cursor is defined.
- Process: Removes the element currently pointed to by the cursor. The cursor is advanced to the next element (or resets to the first element if the last element was deleted).
- Post-condition: List size decreases by 1.

empty(): boolean

- Input: None.
- Process: Checks if the list contains any elements.
- Output: Returns true if the list size is 0, false otherwise.

last(): boolean

- Input: None.
- Requires: The cursor is defined.
- Process: Checks whether the cursor is currently pointing to the final node of the sequence.

- Output: Returns true if at the last node, false otherwise.

]

3. Design

[The Student Advising System is designed using core object-oriented programming principles to create a clean, modular separation between data storage and system functionality. The entire architecture relies on an interface-driven layout, meaning the user execution menu communicates with the system purely through predefined behavioral contracts. This keeps the components decoupled, allowing the underlying data structures to be modified or upgraded in the future without breaking the main application logic.

At the foundation of the system is a custom-built, generic Singly LinkedList that operates using an internal cursor state strategy, maintaining a head pointer and a current pointer reference. Conceptually, this behaves like a sequential train of files where a specific "bookmark" (the cursor) tracks the active record. Because the built-in Java Collections Framework is completely avoided to satisfy system constraints, any search, insertion, or deletion requires the system to systematically move this cursor node-by-node from the front of the list. Once the desired position is found, structural changes are performed smoothly in constant time by shifting local node pointers.

System Component Blueprints

Component	Role & Structural Design	Technical Character
Node and LinkedList	The foundational data structure layer that encapsulates generic data and sequential node references.	Custom data structure running on internal cursor states (head and current pointers).
Student	Represents individual profiles. Holds personal attributes and wraps an internal LinkedList to maintain a personal schedule.	Implements the IStudent interface.
Event	The parent polymorphic blueprint holding universal attributes like ID, title, location, and timestamps.	Abstract Class implementing the IEvent interface.
Meeting	Represents a 1-to-1 advisor-student consultation, holding a direct pointer reference to a single Student node.	Subclass of Event implementing IMeeting.

Component	Role & Structural Design	Technical Character
Workshop	Represents a 1-to-many group event, holding an internal LinkedList to track multiple dynamic participants.	Subclass of Event implementing IWorkshop.
AdvisingSystem	The master controller of the application that coordinates background CSV file parsing and schedule binding.	Implements the IAdvisingSystem interface.

The entities within this design interact dynamically while protecting data integrity through a chronological conflict guard. Each student manages their own encapsulated list of events, and the system leverages the Comparable contract built into the DateTime entity to prevent overlapping appointments. Before any individual Meeting or group Workshop is officially added to a student's schedule, the system performs a validation sweep through their existing events. If a time overlap is detected, the operation safely flags the conflict and halts, ensuring reliable schedule coordination.

]

4. Implementation

[The system is implemented using clean object-oriented code, focusing on manual pointer manipulation and chronological data validation:

- **Cursor-Based List:** The LinkedList class manages data sequentially using head and current node references. Operations like insert and remove modify these node pointers directly to adjust memory links in $O(1)$ time once positioned.
- **Polymorphic Events:** The abstract Event class serves as a base blueprint. The Meeting class extends it to hold a single Student reference, while the Workshop class embeds a nested LinkedList to manage multiple dynamic student profiles.
- **Conflict Prevention:** The DateTime class implements the Comparable interface, allowing the system to perform linear time-overlap sweeps through a student's existing schedule before confirming any new booking.

]

5. Performance analysis

[The theoretical performance of data retrieval in the Student Advising System is evaluated using Big-Oh notation to measure efficiency as data volume grows.

- **Before Optimization (Current Implementation):** Without an indexing structure, lookup methods such as `searchStudentById()` or `printEventDetailsByTitle()` must perform a linear scan over the custom `LinkedList` to find a match. If the system contains N records, it sequentially checks every single node from head to tail. This results in a worst-case time complexity of $O(N)$. As the number of students or events scales up, the retrieval time increases linearly.
- **After Optimization (Using an Index Structure):** Implementing an indexing mechanism maps specific search keys (such as Student IDs or Event Titles) directly to their memory references. Instead of traversing the entire `LinkedList`, the system can instantly jump to the target record. The lookup operation takes $O(1)$ constant time using a hash structure (or $O(\log N)$ using a balanced tree layer). This optimizes the retrieval bounds to $O(1)$, ensuring high-speed queries regardless of how large the advising dataset becomes.

]

6. Conclusion

[Phase 1 of the Student Advising System was successfully implemented by building a reliable back-end system using a custom cursor-based singly linked list. The project achieved its main goals of cleanly parsing bulk data from CSV files, managing student profiles, and scheduling conflict-free individual meetings and collective workshops, while strictly complying with the constraint of avoiding built-in Java collections.

Limitations and Future Work:

The primary limitation of the current design is its reliance on linear data structures, which require sequential scanning. While global lookup tasks remain linear $O(N)$, validation tasks such as conflict checking have been optimized to $O(M)$, where M is the number of events in a specific student's schedule. However, relying on linear structures for system-wide searches can still slow down performance when processing extremely large datasets. Future work will focus on upgrading the data management tier to advanced non-linear structures, such as Binary Search Trees (BST) or Inverted Index Hash Maps, to successfully reduce search and retrieval time bounds to $O(\log N)$ or $O(1)$.

]